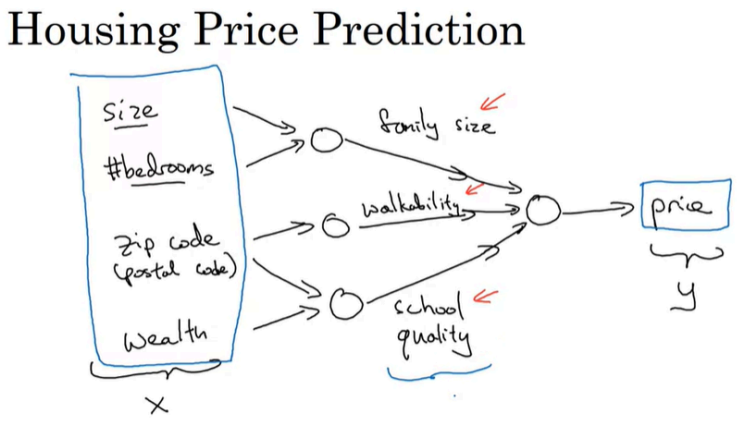
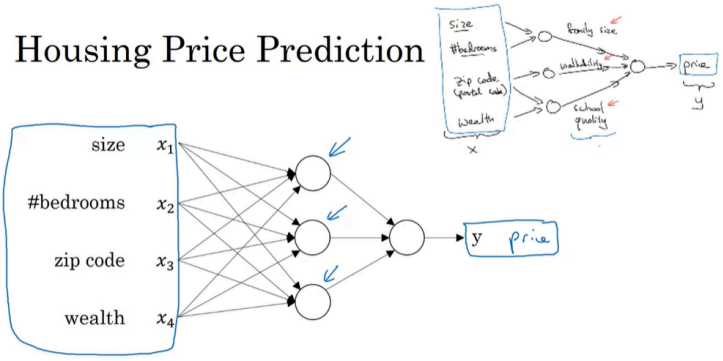
Deep Learning Specialization

Neural Networks and Deep Learning

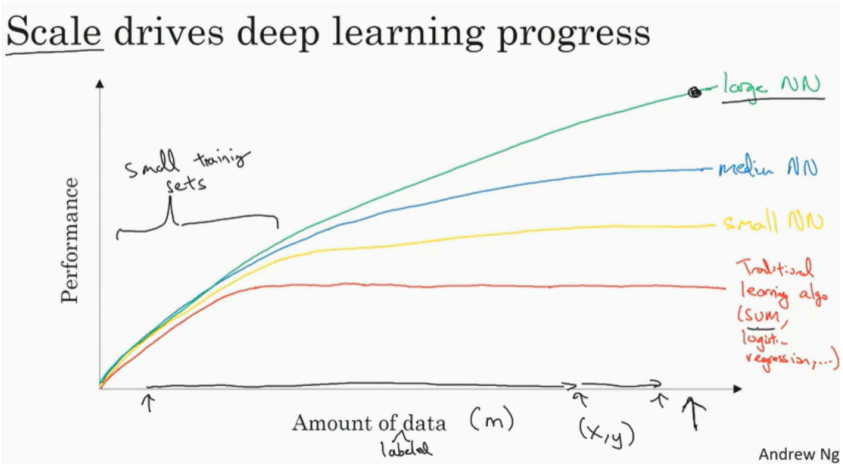
Introduction to Deep Learning

What is a Neural Network?





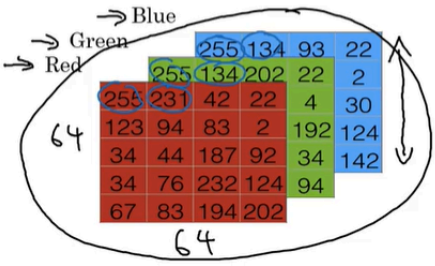
Why is Deep Learning taking off?



Logistic Regression as a Neural Network

Binary Classification

Binary Classification



$$x = \begin{bmatrix} 255 \\ 231 \\ \vdots \\ 255 \\ 134 \\ \vdots \end{bmatrix}$$

$64 \times 64 \times 3 = 12288$

$n = n_x = 12288$

$x \longrightarrow y$

Andrew Ng

Notation

(x, y) $\quad x \in \mathbb{R}^{n_x}$, $y \in \{0, 1\}$

m training examples: $\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \ldots, (x^{(m)}, y^{(m)})\}$

$M = M_{train}$ $\qquad M_{test}$ = #test examples.

$$X = \begin{bmatrix} | & | & & | \\ x^{(1)} & x^{(2)} & \cdots & x^{(m)} \\ | & | & & | \end{bmatrix}$$

n_x, m

$x^{(1)}$, $x^{(m)}$

$X \in \mathbb{R}^{n_x \times m}$ $\qquad X.shape = (n_x, m)$

$$Y = \begin{bmatrix} y^{(1)} & y^{(2)} & \cdots & y^{(m)} \end{bmatrix}$$

$Y \in \mathbb{R}^{1 \times m}$

$Y.shape = (1, m)$

Logistic Regression

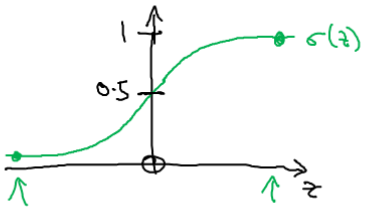
Logistic Regression

Given x, want $\hat{y} = P(y=1|x)$, $0 \le \hat{y} \le 1$

$x \in \mathbb{R}^{n_x}$

Parameters: $w \in \mathbb{R}^{n_x}$, $b \in \mathbb{R}$.

Output $\hat{y} = \sigma(w^T x + b)$, z



$x_0 = 1$, $x \in \mathbb{R}^{n_x+1}$

$\hat{y} = \sigma(\theta^T x)$

$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \vdots \\ \theta_{n_x} \end{bmatrix}$ $\}b \leftarrow$ $\}w \leftarrow$

$\sigma(z) = \frac{1}{1+e^{-z}}$

If z large $\sigma(z) \approx \frac{1}{1+0} = 1$

If z large negative number

$\sigma(z) = \frac{1}{1+e^{-z}} \approx \frac{1}{1+\text{Bignum}} \approx 0$

Andrew N

Logistic Regression Cost Function

Logistic Regression cost function

$\Rightarrow \hat{y}^{(i)} = \sigma(w^T x^{(i)} + b)$, where $\sigma(z^{(i)}) = \frac{1}{1+e^{-z^{(i)}}}$ $\quad z^{(i)} = w^T x^{(i)} + b$

Given $\{(x^{(1)}, y^{(1)}), \ldots, (x^{(m)}, y^{(m)})\}$, want $\hat{y}^{(i)} \approx y^{(i)}$.

$x^{(i)}$, $y^{(i)}$, $z^{(i)}$ i-th example.

Loss (error) function: $\mathcal{L}(\hat{y}, y) = \frac{1}{2}(\hat{y}-y)^2$

$\mathcal{L}(\hat{y}, y) = -(y \log \hat{y} + (1-y)\log(1-\hat{y}))$ $\leftarrow$

If $y=1$: $\mathcal{L}(\hat{y}, y) = -\log \hat{y}$ $\leftarrow$ want $\log \hat{y}$ large, want $\hat{y}$ large.

If $y=0$: $\mathcal{L}(\hat{y}, y) = -\log(1-\hat{y})$ $\leftarrow$ want $\log 1-\hat{y}$ large .... want $\hat{y}$ small

Cost function: $J(w,b) = \frac{1}{m}\sum_{i=1}^{m} \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)} \log \hat{y}^{(i)} + (1-y^{(i)}) \log(1-\hat{y}^{(i)})\right]$

G.D makes it harder for Logistic Algo. to find local minima

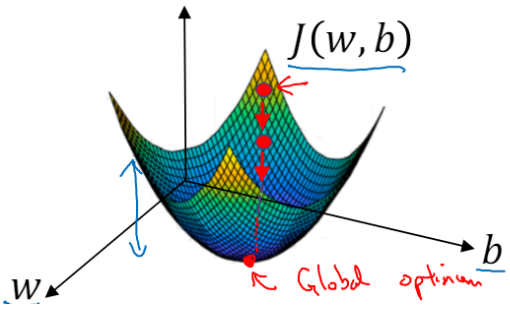
Gradient Descent

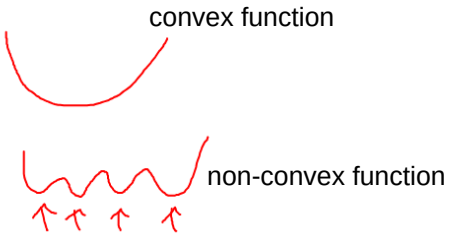
Gradient Descent

Recap: $\hat{y} = \sigma(w^T x + b)$, $\sigma(z) = \frac{1}{1+e^{-z}}$ ←

$$J(w,b) = \frac{1}{m}\sum_{i=1}^{m} \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) = -\frac{1}{m}\sum_{i=1}^{m} y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})$$

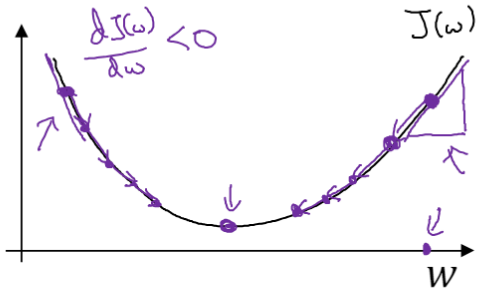
Want to find w, b that minimize $J(w,b)$





So the fact that our cost function J of w, b as defined here is convex, is one of the huge reasons why we use this particular cost function J for logistic regression.

Gradient Descent



Repeat {

$w := w - \alpha \frac{dJ(w)}{dw}$ ("dw")

}

learning rate

$w := w - \alpha dw$

$\frac{dJ(w)}{dw} = ?$

$J(w,b)$

$w := w - \alpha \frac{\partial J(w,b)}{\partial w}$ → $\frac{\partial J(w,b)}{\partial w}$ → dw

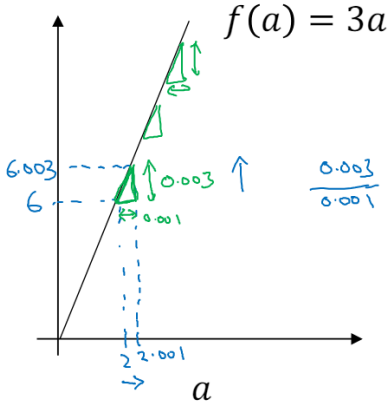
$b := b - \alpha \frac{\partial J(w,b)}{\partial b}$ → $\frac{\partial J(w,b)}{\partial b}$ → db

"partial derivative" J

Andrew Ng

Derivatives

Intuition about derivatives



$\frac{0.003}{0.001}$ $\frac{\text{height}}{\text{width}}$

→ slope (derivative) of $f(a)$ at $a=2$ is 3

$a = 2$ $\quad f(a) = 6$

$a = 2.001$ $\quad f(a) = 6.003$

$a = 5$ $\quad f(a) = 15$

$a = 5.001$ $\quad f(a) = 15.003$

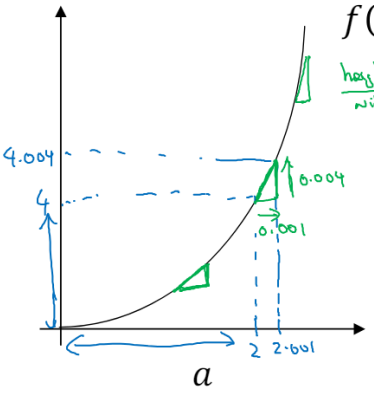
slope at $a=5$ is also 3

$\frac{d}{da} f(a) = 3 = \frac{d}{da} f(a)$

0.001 ←
0.000000001
0.000000000001

More derivatives examples

Intuition about derivatives



$\frac{d}{da} a^2 = 2a$

0.001
$(2a) \times 0.001$

0.001 ←
0.00000....01 ←

$a = 2$ $\quad f(a) = 4$

$a = 2.001$ $\quad f(a) \approx 4.004$ $(4.004\boxed{001})$

slope (derivative) of $f(a)$ at $a=2$ is 4.

$\frac{d}{da} f(a) = 4$ when $a=2$.

$a = 5$ $\quad f(a) = 25$

$a = 5.001$ $\quad f(a) \approx 25.010$

$\frac{d}{da} f(a) = 10$ when $a=5$

$\frac{d}{da} f(a) = \frac{d}{da} a^2 = 2a$

More derivative examples

$f(a) = a^2$ $\quad \frac{d}{da} f(a) = \underbrace{2a}_{4}$

$a = 2$ $\quad f(a) = 4$

$a = 2.001$ $\quad f(a) \approx 4.004$

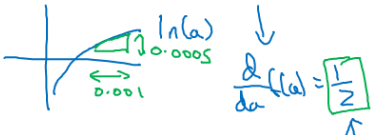
$f(a) = a^3$ $\quad \frac{d}{da} f(a) = \underbrace{3a^2}_{3 \times 2^2 = 12}$

$a = 2$ $\quad f(a) = 8$

$a = 2.001$ $\quad f(a) \approx 8.012$

$f(a) = \log_e(a)$
$\ln(a)$

$\frac{d}{da} f(a) = \frac{1}{a}$



$\frac{d}{da} f(a) = \frac{1}{2}$

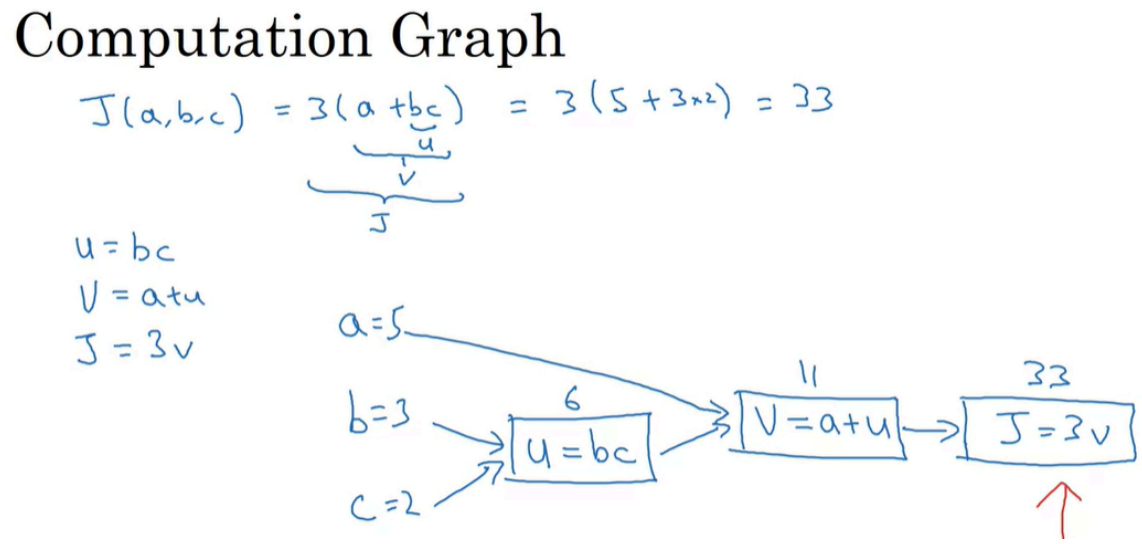
$a = 2$ $\quad f(a) \approx 0.69315$

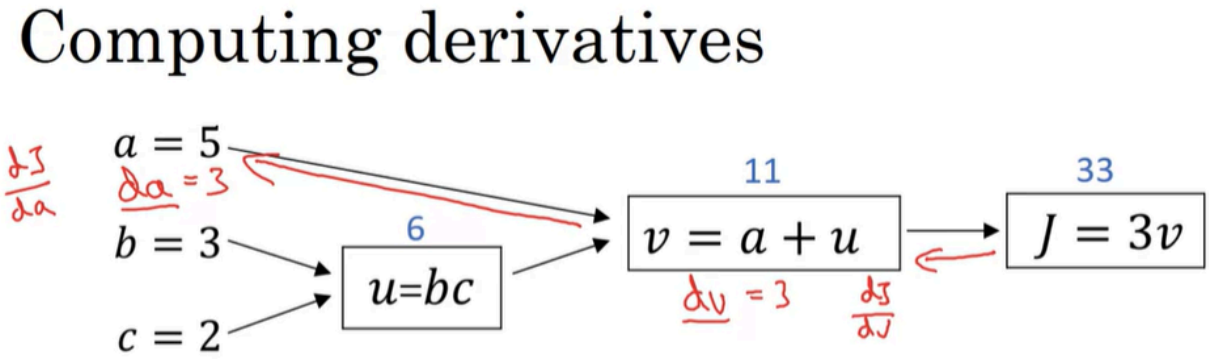
$a = 2.001$ $\quad f(a) \approx 0.69365$

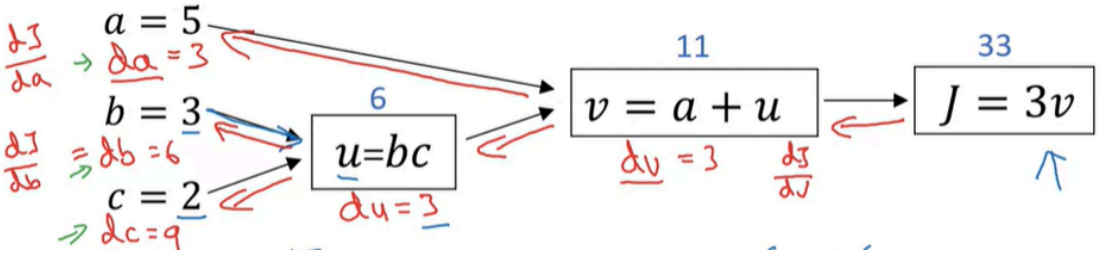
0.0005 ↔ 0.0005

Computation Graph

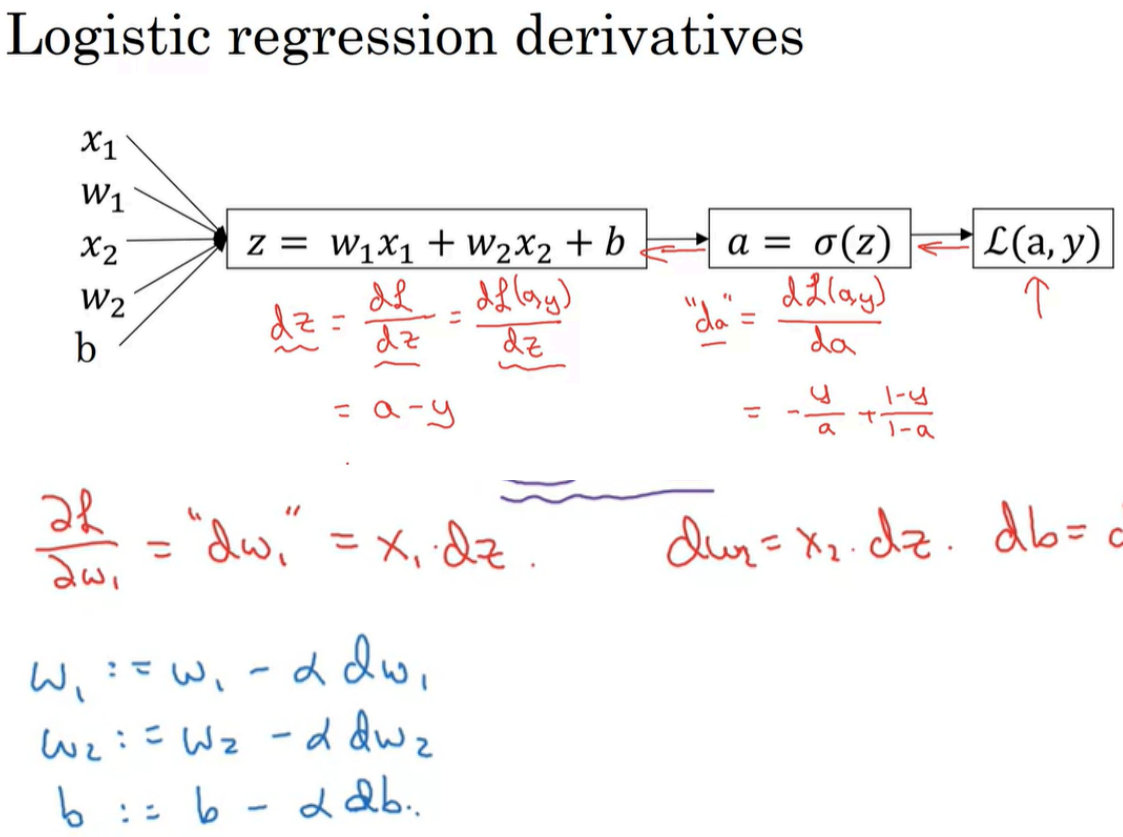
You've heard me say that the computations of a neural network are organized in terms of a forward pass or a forward propagation step, in which we compute the output of the neural network, followed by a backward pass or back propagation step, which we use to compute gradients or compute derivatives. The computation graph explains why it is organized this way.





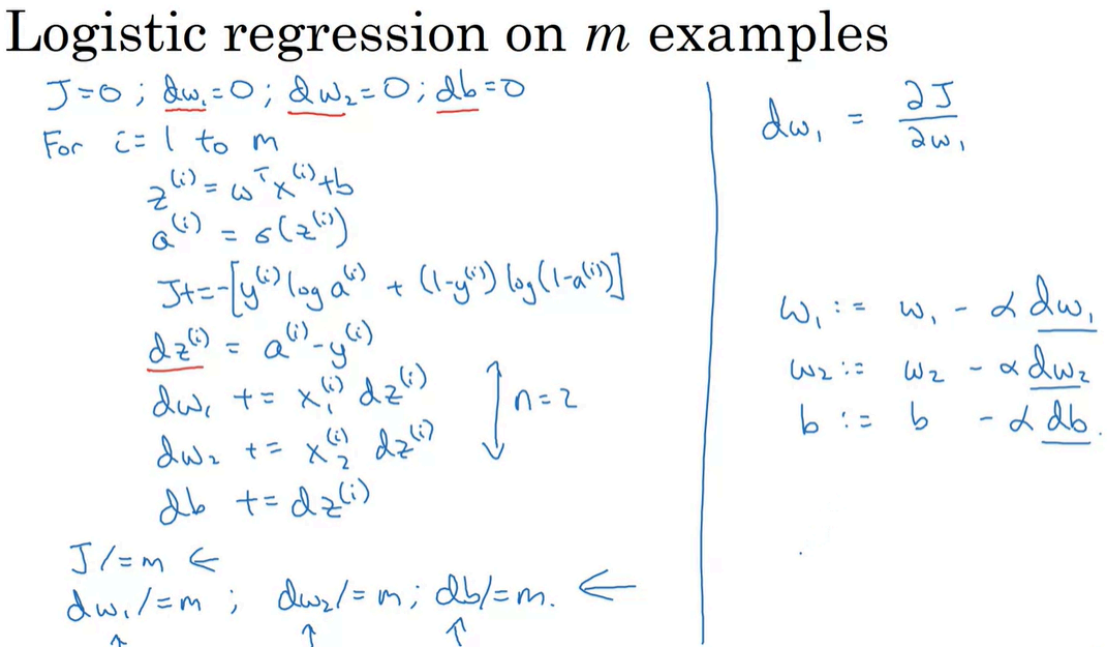


Logistic Regression Gradient Descent



Conclusion: GD moves the value by a single point. Then derivatives change again, we move the value again by that small point and repeat the action.

Gradient Descent on m Examples



Python and Vectorization

Vectorization

What is vectorization?

$z = w^T x + b$

$w = \begin{bmatrix} \vdots \\ \vdots \end{bmatrix} \quad x = \begin{bmatrix} \vdots \\ \vdots \end{bmatrix}$ $\quad w \in \mathbb{R}^{n_x}$ $\quad x \in \mathbb{R}^{n_x}$

Non-vectorized:

```
z = 0
for i in range(n-x):
    z += w[i] * x[i]
z += b
```

Vectorized

```
z = np.dot(w,x) + b
```

$w^T x$

→ GPU
→ CPU
} SIMD - single instruction multiple data.

More Vectorization Examples

Neural network programming guideline

Whenever possible, avoid explicit for-loops.

Logistic regression derivatives

```
J = 0, dw1 = 0, dw2 = 0, db = 0
for i = 1 to m:
    z(i) = w^T x(i) + b
    a(i) = σ(z(i))
    J += -[y(i) log a(i) + (1 - y(i)) log(1 - a(i))]
    dz(i) = a(i) - y(i)
    dw1 += x1(i) dz(i)
    dw2 += x2(i) dz(i)
    db += dz(i)
J = J/m, dw1 = dw1/m, dw2 = dw2/m, db = db/m
```

$dw = np.zeros((n_x, 1))$

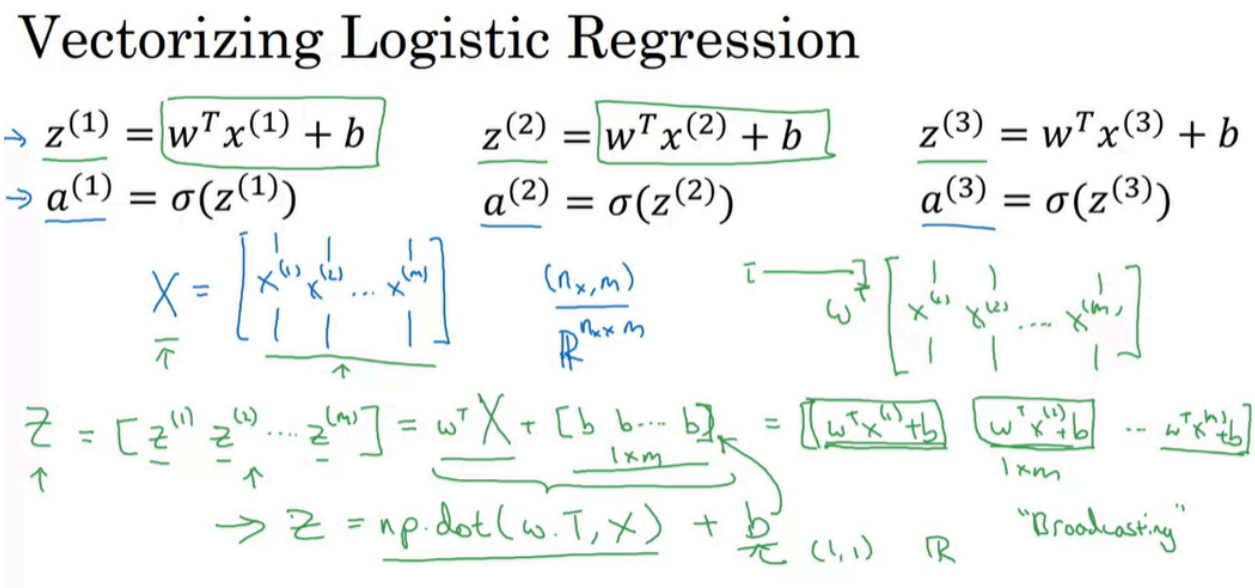
for $j = 1 \ldots n_x$: $dw_j += \ldots$

$n_x = 2$

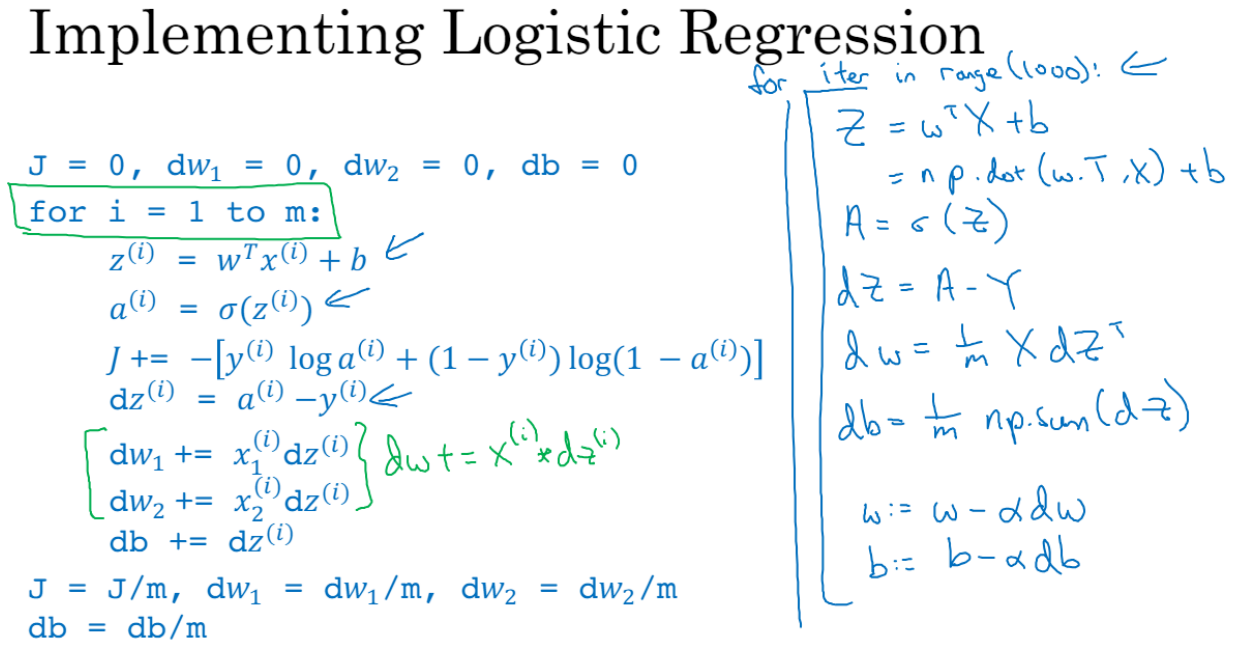
$dw += x^{(i)} dz^{(i)}$

$dw /= m$.

Vectorizing Logistic Regression



Vectorizing Logistic Regression's Gradient Output



Broadcasting in Python

Broadcasting example

Calories from Carbs, Proteins, Fats in 100g of different foods:

	Apples	Beef	Eggs	Potatoes
Carb	56.0	0.0	4.4	68.0
Protein	1.2	104.0	52.0	8.0
Fat	1.8	135.0	99.0	0.9

$= A$ (3,4)

59cal $\frac{56}{59} \approx 94.9\%$

Calculate % of calories from Carb, Protein, Fat. Can you do this without explicit for-loop?

```
cal = A.sum(axis = 0)
percentage = 100*A/(cal.reshape(1,4))
```

(3,4) / (1,4)

$$\begin{bmatrix}1\\2\\3\\4\end{bmatrix} + \begin{bmatrix}100\\100\\100\\100\end{bmatrix} 100 = \begin{bmatrix}101\\102\\103\\104\end{bmatrix}$$

$$\begin{bmatrix}1 & 2 & 3\\4 & 5 & 6\end{bmatrix} + \begin{bmatrix}100 & 200 & 300\end{bmatrix} = \begin{bmatrix}101 & 202 & 303\\104 & 205 & 306\end{bmatrix}$$

(m,n) (2,3) (1,n) ⇝ (m,n) (2,3)

$$\begin{bmatrix}1 & 2 & 3\\4 & 5 & 6\end{bmatrix} + \begin{bmatrix}100\\200\end{bmatrix} = \begin{bmatrix}101 & 102 & 103\\204 & 205 & 206\end{bmatrix}$$

(m,n) (m,1) ⇝ (m,n)

A Note on Python/Numpy Vectors

Python/numpy vectors

```
a = np.random.randn(5)
```

a.shape = (5,)

"rank 1 array"

} Don't use

```
a = np.random.randn(5,1)
```

→ a.shape = (5,1) column vector

```
a = np.random.randn(1,5)
```

→ a.shape = (1,5) row vector

```
assert(a.shape == (5,1))
```

←

a = a.reshape((5,1))